

编译原理

第三讲：词法分析

授课教师：柯嵩宇

福州大学计算机与大数据学院

2026 年 3 月 12 日



福州大学
FUZHOU UNIVERSITY 1958



- ① 知识点回顾
- ② 词法分析
- ③ 手写一个词法分析器
- ④ 小结与作业

目录

① 知识点回顾

编译过程的步骤

编程语言与文法

② 词法分析

③ 手写一个词法分析器

④ 小结与作业

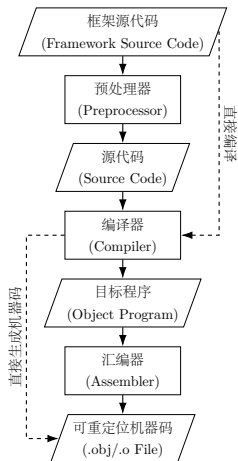


目录

- ① 知识点回顾
编译过程的步骤
编程语言与文法
- ② 词法分析
- ③ 手写一个词法分析器
- ④ 小结与作业



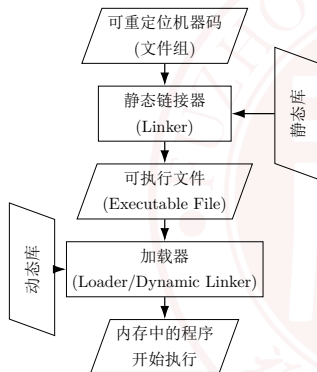
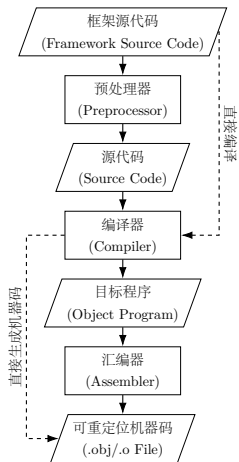
编译系统的构成



单个文件的编译流程

- 预处理：处理文件导入、宏定义等
- 编译：将源代码转换为目标代码，通常为目标的汇编指令
- 汇编：将汇编指令转换为机器代码，输出为可重定位的目标文件（Relocatable Object File）
- 对于某些语言或者部分文件，可以跳过预处理和汇编阶段
- **问题：一个程序有 100000 行代码，如何高效地编译？**

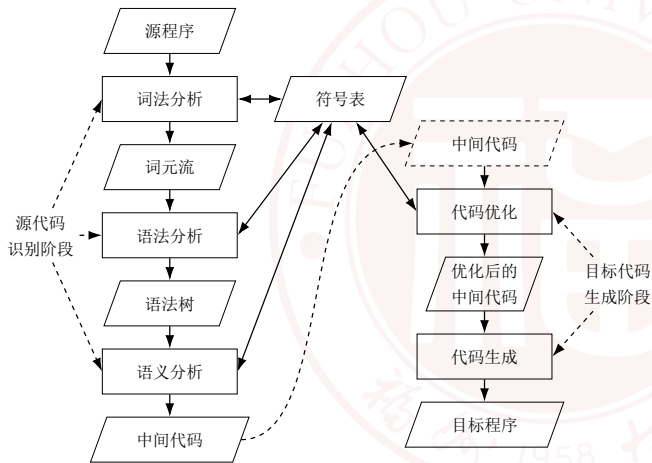
编译系统的构成



现代编译后续流程

- 静态链接程序 (Static Linker): 将多个目标文件和库文件链接为单个可执行文件
- 动态链接程序 (Dynamic Linker): 在运行时将共享库加载到内存中并处理好跳转

现代编译器的主要阶段



目录

① 知识点回顾

编译过程的步骤
编程语言与文法

② 词法分析

③ 手写一个词法分析器

④ 小结与作业



程序设计语言的三要素

- 语法

- 程序的结构形式
- 代码的书写规则
- 符号的组合方式
- 决定了程序是否“合法”



程序设计语言的三要素

• 语法

- 程序的结构形式
- 代码的书写规则
- 符号的组合方式
- 决定了程序是否“合法”

• 语义

- 程序的含义
- 代码的执行逻辑
- 表达式的计算结果
- 决定了程序“做什么”

程序设计语言的三要素

• 语法

- 程序的结构形式
- 代码的书写规则
- 符号的组合方式
- 决定了程序是否“合法”

• 语义

- 程序的含义
- 代码的执行逻辑
- 表达式的计算结果
- 决定了程序“做什么”

• 标准库

- 语言提供的基础功能
- 常用的数据结构和算法
- 系统接口和工具函数
- 决定了开发效率

文法

- 语法（grammar，也称文法）是任意自然语言中句子、短语以及词等语法单位的语法结构与语法意义的规律，本质上即音义结合体之间的结合规律。
- 形式文法：在形式语言理论中，文法（formal grammar）是形式语言中字符串的一套产生式规则（production rule）。
- 形式文法描述了如何用语言的字母表生成符合句法（syntax）的有效的字符串。
- 文法不描述字符串的含义，也不描述在任何上下文中可以用它们做什么——只描述它们的形式。

形式文法

定义

一个文法可以用一个四元组描述： $G = (V, \Sigma, P, S)$ ，其中

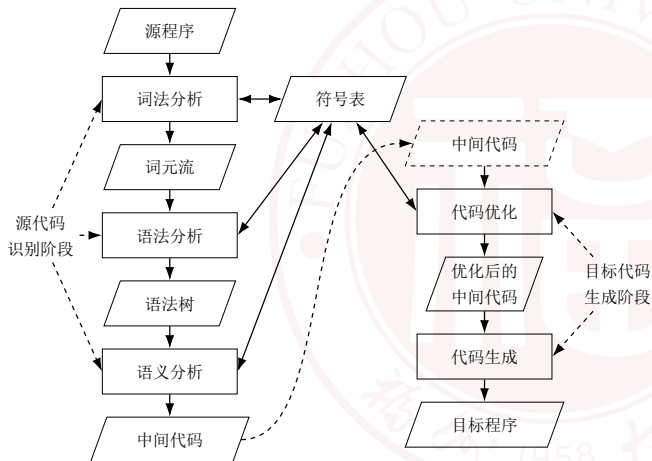
- V 是非终结符的集合
- Σ 是终结符集合，即字符集
- P 是产生式的集合
- 产生式是形如 $S \rightarrow abc$ 式子，表示可以将一个串的中左侧部分替换为右侧部分
- S 是起始符号（Initial Symbol，亦称初始符号），决定产生式的替换由哪个符号开始

形式文法的层次

乔姆斯基体系是计算机科学中刻画形式文法表达能力的一个分类谱系，是由语言学家诺姆·乔姆斯基于 1956 年提出的。它包括四个层次：

- **0-型文法（无限制文法或短语结构文法）**：包括所有的文法。
- **1-型文法（上下文相关文法）**：生成上下文相关语言。这种文法的产生式规则取如 $\alpha A \beta \rightarrow \alpha \gamma \beta$ 一样的形式。
- **2-型文法（上下文无关文法）**：生成上下文无关语言。这种文法的产生式规则取如 $A \rightarrow \gamma$ 一样的形式。
- **3-型文法（正则文法）**：生成正则语言。产生式的左侧只能包含一个非终结符号，产生式的右侧只能是空串、一个终结符号或者一个终结符号后随一个非终结符号。

现代编译器的主要阶段



目录

① 知识点回顾

② 词法分析

词法分析的任务

为什么要做词法分析？

两种词法分析设计思路

③ 手写一个词法分析器

④ 小结与作业



目录

① 知识点回顾

② 词法分析

词法分析的任务

为什么要做词法分析？

两种词法分析设计思路

③ 手写一个词法分析器

④ 小结与作业



词法分析器的功能

- 词法分析器的主要功能是从左到右扫描构成源程序的字符流，识别出一个个具有独立意义的单词，并转换为统一的内部表示。
 - 输入处理：读取源程序的字符流（通常从源文件或缓冲区中）。

词法分析器的功能

- 词法分析器的主要功能是从左到右扫描构成源程序的字符流，识别出一个个具有独立意义的单词，并转换为统一的内部表示。
 - ① 输入处理：读取源程序的字符流（通常从源文件或缓冲区中）。
 - ② 识别单词：根据源语言的词法规则（即构词规则），识别出关键字、标识符、运算符、界符和常量等。

词法分析器的功能

- 词法分析器的主要功能是从左到右扫描构成源程序的字符流，识别出一个个具有独立意义的单词，并转换为统一的内部表示。
 - ① 输入处理：读取源程序的字符流（通常从源文件或缓冲区中）。
 - ② 识别单词：根据源语言的词法规则（即构词规则），识别出关键字、标识符、运算符、界符和常量等。
 - ③ 输出 Token 序列：将识别出的单词以词法单元的形式输出给语法分析器。一个 Token 通常包含两个部分：

词法分析器的功能

- 词法分析器的主要功能是从左到右扫描构成源程序的字符流，识别出一个个具有独立意义的单词，并转换为统一的内部表示。
 - ① 输入处理：读取源程序的字符流（通常从源文件或缓冲区中）。
 - ② 识别单词：根据源语言的词法规则（即构词规则），识别出关键字、标识符、运算符、界符和常量等。
 - ③ 输出 Token 序列：将识别出的单词以词法单元的形式输出给语法分析器。一个 Token 通常包含两个部分：
 - 单词类别：如关键字、标识符、常数等（通常用整数编码表示）。

词法分析器的功能

- 词法分析器的主要功能是从左到右扫描构成源程序的字符流，识别出一个个具有独立意义的单词，并转换为统一的内部表示。
 - ① 输入处理：读取源程序的字符流（通常从源文件或缓冲区中）。
 - ② 识别单词：根据源语言的词法规则（即构词规则），识别出关键字、标识符、运算符、界符和常量等。
 - ③ 输出 Token 序列：将识别出的单词以词法单元的形式输出给语法分析器。一个 Token 通常包含两个部分：
 - 单词类别：如关键字、标识符、常数等（通常用整数编码表示）。
 - 单词属性：单词的具体值（如标识符的名称、常数的数值）。

词法分析器的功能

- 词法分析器的主要功能是从左到右扫描构成源程序的字符流，识别出一个个具有独立意义的单词，并转换为统一的内部表示。
 - ① 输入处理：读取源程序的字符流（通常从源文件或缓冲区中）。
 - ② 识别单词：根据源语言的词法规则（即构词规则），识别出关键字、标识符、运算符、界符和常量等。
 - ③ 输出 Token 序列：将识别出的单词以词法单元的形式输出给语法分析器。一个 Token 通常包含两个部分：
 - 单词类别：如关键字、标识符、常数等（通常用整数编码表示）。
 - 单词属性：单词的具体值（如标识符的名称、常数的数值）。
 - ④ 建立符号表接口：将标识符填入符号表，并记录其相关信息，为后续的语义分析做准备。

词法分析器的具体任务

为了实现上述功能，词法分析器需要完成一系列具体的、细致的工作任务：

① 滤除无用成分：



词法分析器的具体任务

为了实现上述功能，词法分析器需要完成一系列具体的、细致的工作任务：

① 滤除无用成分：

- 删除源程序中的注释。



词法分析器的具体任务

为了实现上述功能，词法分析器需要完成一系列具体的、细致的工作任务：

① 滤除无用成分：

- 删除源程序中的注释。
- 跳过空格、制表符、换行符等空白字符，这些字符通常仅用于提高代码可读性，对程序执行无意义。

词法分析器的具体任务

为了实现上述功能，词法分析器需要完成一系列具体的、细致的工作任务：

① 滤除无用成分：

- 删除源程序中的注释。
- 跳过空格、制表符、换行符等空白字符，这些字符通常仅用于提高代码可读性，对程序执行无意义。

② 识别并归类单词：

词法分析器的具体任务

为了实现上述功能，词法分析器需要完成一系列具体的、细致的工作任务：

① 滤除无用成分：

- 删除源程序中的注释。
- 跳过空格、制表符、换行符等空白字符，这些字符通常仅用于提高代码可读性，对程序执行无意义。

② 识别并归类单词：

- 根据语言的词法规则，将字符流分割成独立的单词。

词法分析器的具体任务

为了实现上述功能，词法分析器需要完成一系列具体的、细致的工作任务：

① 滤除无用成分：

- 删除源程序中的注释。
- 跳过空格、制表符、换行符等空白字符，这些字符通常仅用于提高代码可读性，对程序执行无意义。

② 识别并归类单词：

- 根据语言的词法规则，将字符流分割成独立的单词。
- 判断单词的类型（例如：判断 `if` 是关键字，而 `i1` 是标识符）。

词法分析器的具体任务

为了实现上述功能，词法分析器需要完成一系列具体的、细致的工作任务：

① 滤除无用成分：

- 删除源程序中的注释。
- 跳过空格、制表符、换行符等空白字符，这些字符通常仅用于提高代码可读性，对程序执行无意义。

② 识别并归类单词：

- 根据语言的词法规则，将字符流分割成独立的单词。
- 判断单词的类型（例如：判断 if 是关键字，而 i1 是标识符）。
- 对不同类型的单词进行分类编码。

词法分析器的具体任务

为了实现上述功能，词法分析器需要完成一系列具体的、细致的工作任务：

① 滤除无用成分：

- 删除源程序中的注释。
- 跳过空格、制表符、换行符等空白字符，这些字符通常仅用于提高代码可读性，对程序执行无意义。

② 识别并归类单词：

- 根据语言的词法规则，将字符流分割成独立的单词。
- 判断单词的类型（例如：判断 if 是关键字，而 i1 是标识符）。
- 对不同类型的单词进行分类编码。

③ 管理符号表：

要完成一系列具体的、细致的工作任务：

空白字符，这些字符通常仅用于提高代码可读性

流分割成独立的单词。

if 是关键字，而 i1 是标识符)。

码。

表中是否已存在该条目。

- 当遇到标识符时，检查符号表中是否已存在该条目。

词法分析器的具体任务

为了实现上述功能，词法分析器需要完成一系列具体的、细致的工作任务：

① 滤除无用成分：

- 删除源程序中的注释。
- 跳过空格、制表符、换行符等空白字符，这些字符通常仅用于提高代码可读性，对程序执行无意义。

② 识别并归类单词：

- 根据语言的词法规则，将字符流分割成独立的单词。
- 判断单词的类型（例如：判断 if 是关键字，而 i1 是标识符）。
- 对不同类型的单词进行分类编码。

③ 管理符号表：

- 当遇到标识符时，检查符号表中是否已存在该条目。
- 如果不存在，则在符号表中为该标识符建立新的条目。

词法分析器的具体任务

为了实现上述功能，词法分析器需要完成一系列具体的、细致的工作任务：

① 滤除无用成分：

- 删除源程序中的注释。
- 跳过空格、制表符、换行符等空白字符，这些字符通常仅用于提高代码可读性，对程序执行无意义。

② 识别并归类单词：

- 根据语言的词法规则，将字符流分割成独立的单词。
- 判断单词的类型（例如：判断 if 是关键字，而 i1 是标识符）。
- 对不同类型的单词进行分类编码。

③ 管理符号表：

- 当遇到标识符时，检查符号表中是否已存在该条目。
- 如果不存在，则在符号表中为该标识符建立新的条目。
- 将指向符号表条目的指针作为属性值附加到 Token 中。

词法分析器的具体任务（续）

④ 处理字面量：



词法分析器的具体任务（续）

④ 处理字面量：

- 识别数字常量（整数、浮点数），并将其从字符串形式转换为机器内部表示（如二进制数值）。



词法分析器的具体任务（续）

④ 处理字面量：

- 识别数字常量（整数、浮点数），并将其从字符串形式转换为机器内部表示（如二进制数值）。
- 识别字符串常量，并处理其中的转义字符（如 `\n`）。

词法分析器的具体任务（续）

④ 处理字面量：

- 识别数字常量（整数、浮点数），并将其从字符串形式转换为机器内部表示（如二进制数值）。
- 识别字符串常量，并处理其中的转义字符（如 `\n`）。

⑤ 错误检测与报告：

词法分析器的具体任务（续）

④ 处理字面量：

- 识别数字常量（整数、浮点数），并将其从字符串形式转换为机器内部表示（如二进制数值）。
- 识别字符串常量，并处理其中的转义字符（如 `\n`）。

⑤ 错误检测与报告：

- 检测源程序中的词法错误。最常见的词法错误包括：

词法分析器的具体任务（续）

④ 处理字面量：

- 识别数字常量（整数、浮点数），并将其从字符串形式转换为机器内部表示（如二进制数值）。
- 识别字符串常量，并处理其中的转义字符（如 `\n`）。

⑤ 错误检测与报告：

- 检测源程序中的词法错误。最常见的词法错误包括：
 - 非法字符（源语言字符集中不存在的字符）。

词法分析器的具体任务（续）

④ 处理字面量：

- 识别数字常量（整数、浮点数），并将其从字符串形式转换为机器内部表示（如二进制数值）。
- 识别字符串常量，并处理其中的转义字符（如 `\n`）。

⑤ 错误检测与报告：

- 检测源程序中的词法错误。最常见的词法错误包括：
 - 非法字符（源语言字符集中不存在的字符）。
 - 字符串或注释缺少结束符。

词法分析器的具体任务（续）

④ 处理字面量：

- 识别数字常量（整数、浮点数），并将其从字符串形式转换为机器内部表示（如二进制数值）。
- 识别字符串常量，并处理其中的转义字符（如 `\n`）。

⑤ 错误检测与报告：

- 检测源程序中的词法错误。最常见的词法错误包括：
 - 非法字符（源语言字符集中不存在的字符）。
 - 字符串或注释缺少结束符。
 - 数字常量书写格式错误（如 `123abc`，虽然有时这会触发词法/语法边界问题）。

词法分析器的具体任务（续）

④ 处理字面量：

- 识别数字常量（整数、浮点数），并将其从字符串形式转换为机器内部表示（如二进制数值）。
- 识别字符串常量，并处理其中的转义字符（如 `\n`）。

⑤ 错误检测与报告：

- 检测源程序中的词法错误。最常见的词法错误包括：
 - 非法字符（源语言字符集中不存在的字符）。
 - 字符串或注释缺少结束符。
 - 数字常量书写格式错误（如 `123abc`，虽然有时这会触发词法/语法边界问题）。
- 记录错误的位置（行号、列号），并向用户报告。

目录



① 知识点回顾

② 词法分析

词法分析的任务

为什么要做词法分析?

两种词法分析设计思路

③ 手写一个词法分析器

④ 小结与作业

为什么要做词法分析?

- 现代编程语言的词法通常是基于正则文法的，而语法分析器通常是基于上下文无关文法的。正则文法是上下文无关文法的一个真子集。
- 所以，能不能直接用语法分析器来处理词法分析的任务？
- 理论上是可以的，但实际上会带来很多麻烦。

基于字符的语法分析的缺点

① 语法规则急剧膨胀

语法分析器必须定义所有单词的构成规则，例如：

- 标识符：letter (letter | digit)*
- 数字：digit+ ('.' digit+)?
- 关键字：if、while 等。

绝大多数词法规则是正则的，强行用上下文无关文法描述是低效的。

基于字符的语法分析的缺点

① 语法规则急剧膨胀

语法分析器必须定义所有单词的构成规则，例如：

- 标识符: `letter (letter | digit)*`
- 数字: `digit+ ('.' digit+)?`
- 关键字: `if`、`while` 等。

绝大多数词法规则是正则的，强行用上下文无关文法描述是低效的。

② 处理空白和注释的复杂性

空白和注释在源文件中几乎无处不在，如果由语法分析器处理，则必须在每条产生式中显式地考虑它们的位置。例如，一个表达式文法可能变成：

- `expr → term { ('+' | '-') term }`
- `term → factor { ('*' | '/') factor }`
- `factor → '(' whitespace expr whitespace ')' | number`
- `whitespace → ' ' | '\t' | '\n' | comment *`

这导致文法臃肿、易错且难以阅读。

基于字符的语法分析的缺点（续）

③ 效率低下



基于字符的语法分析的缺点（续）

③ 效率低下

- 海量细粒度的字符单元 → 分析树或状态栈爆表 → 大量内存和时间。



基于字符的语法分析的缺点（续）

③ 效率低下

- 海量细粒度的字符单元 → 分析树或状态栈爆表 → 大量内存和时间。

- `a = b + c;`



基于字符的语法分析的缺点（续）

③ 效率低下

- 海量细粒度的字符单元 → 分析树或状态栈爆表 → 大量内存和时间。
 - $a = b + c;$
 - 字符版：至少 12 个字符



基于字符的语法分析的缺点（续）

③ 效率低下

- 海量细粒度的字符单元 → 分析树或状态栈爆表 → 大量内存和时间。
 - $a = b + c;$
 - 字符版：至少 12 个字符
 - Token 版：节点只有 5 个（标识符 a 、运算符 $=$ 、标识符 b 、运算符 $+$ 、标识符 c 和分号;）。

基于字符的语法分析的缺点（续）

③ 效率低下

- 海量细粒度的字符单元 → 分析树或状态栈爆表 → 大量内存和时间。
 - $a = b + c;$
 - 字符版：至少 12 个字符
 - Token 版：节点只有 5 个（标识符 a 、运算符 $=$ 、标识符 b 、运算符 $+$ 、标识符 c 和分号;）。
- 字符级别的分析还可能导致频繁的回溯，严重影响编译速度。

基于字符的语法分析的缺点（续）

③ 效率低下

- 海量细粒度的字符单元 → 分析树或状态栈爆表 → 大量内存和时间。
 - `a = b + c;`
 - 字符版：至少 12 个字符
 - Token 版：节点只有 5 个（标识符 `a`、运算符 `=`、标识符 `b`、运算符 `+`、标识符 `c` 和分号 `;`）。
- 字符级别的分析还可能导致频繁的回溯，严重影响编译速度。

④ 错误诊断困难：词法错误（如字符串缺少结束符）在字符级别处理时，可能被误解为语法结构错误，增加了错误恢复的难度。

基于字符的语法分析的缺点（续）

③ 效率低下

- 海量细粒度的字符单元 → 分析树或状态栈爆表 → 大量内存和时间。
 - $a = b + c;$
 - 字符版：至少 12 个字符
 - Token 版：节点只有 5 个（标识符 a 、运算符 $=$ 、标识符 b 、运算符 $+$ 、标识符 c 和分号;）。
- 字符级别的分析还可能导致频繁的回溯，严重影响编译速度。

④ 错误诊断困难：词法错误（如字符串缺少结束符）在字符级别处理时，可能被误解为语法结构错误，增加了错误恢复的难度。

⑤ 破坏模块化：耦合度过高、不利于团队协作和工具复用。

基于字符的语法分析的缺点（续）

③ 效率低下

- 海量细粒度的字符单元 → 分析树或状态栈爆表 → 大量内存和时间。
 - $a = b + c;$
 - 字符版：至少 12 个字符
 - Token 版：节点只有 5 个（标识符 a 、运算符 $=$ 、标识符 b 、运算符 $+$ 、标识符 c 和分号 $;$ ）。
- 字符级别的分析还可能导致频繁的回溯，严重影响编译速度。

④ 错误诊断困难：词法错误（如字符串缺少结束符）在字符级别处理时，可能被误解为语法结构错误，增加了错误恢复的难度。

⑤ 破坏模块化：耦合度过高、不利于团队协作和工具复用。

⑥ 难以处理语言变体和方言

语法结构相似，但词法规则差异较大（如大小写敏感、注释风格等）。

独立词法分析器可方便替换支持语言变体，而语法分析器可以保持不变。

单独做词法分析的优点

- 简化语法分析：将字符流抽象为 Token，文法更简洁，易于设计。



单独做词法分析的优点

- 简化语法分析：将字符流抽象为 Token，文法更简洁，易于设计。
- 提高效率：Token 数量少，基于有限自动机快速识别，节省资源。

单独做词法分析的优点

- 简化语法分析：将字符流抽象为 Token，文法更简洁，易于设计。
- 提高效率：Token 数量少，基于有限自动机快速识别，节省资源。
- 模块化：词法与语法分离，可独立修改，复用工具（如 Lex/Yacc）。

单独做词法分析的优点

- 简化语法分析：将字符流抽象为 Token，文法更简洁，易于设计。
- 提高效率：Token 数量少，基于有限自动机快速识别，节省资源。
- 模块化：词法与语法分离，可独立修改，复用工具（如 Lex/Yacc）。
- 过滤无关成分：自动忽略空白、注释，减轻语法分析负担。

单独做词法分析的优点

- 简化语法分析：将字符流抽象为 Token，文法更简洁，易于设计。
- 提高效率：Token 数量少，基于有限自动机快速识别，节省资源。
- 模块化：词法与语法分离，可独立修改，复用工具（如 Lex/Yacc）。
- 过滤无关成分：自动忽略空白、注释，减轻语法分析负担。
- 支持预处理与工具：便于实现宏、条件编译，以及语法高亮、静态分析等。

目录

① 知识点回顾

② 词法分析

词法分析的任务

为什么要做词法分析？

两种词法分析设计思路

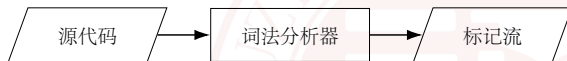
③ 手写一个词法分析器

④ 小结与作业



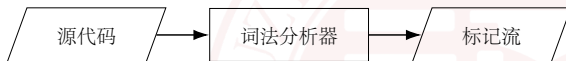
两种词法分析设计思路

① 独立程序型：

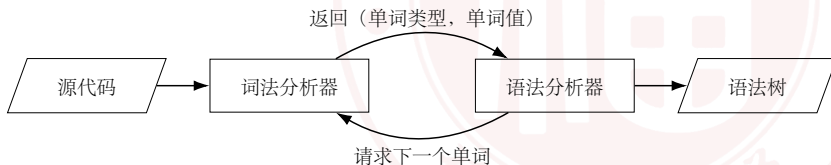


两种词法分析设计思路

① 独立程序型：



② 程序接口型：



目录

① 知识点回顾

② 词法分析

③ 手写一个词法分析器

输入与输出

何时分割单词

单词的类别

词法分析程序的状态与转移

④ 小结与作业



目录

① 知识点回顾

② 词法分析

③ 手写一个词法分析器

输入与输出

何时分割单词

单词的类别

词法分析程序的状态与转移

④ 小结与作业



输入与输出：文件编码与程序内码

- 词法分析器的输入是源代码文件，输出是 Token 序列
- 关键问题：非 ASCII 字符（如中文、日文、韩文）在文件存储与程序处理有一定区别

文件编码（File Encoding）

- 源代码文件在磁盘上存储时使用的编码
- 常见编码：ASCII, UTF-8, GBK, ISO-8859-1
- 示例：中文“中”在 UTF-8 编码下存储为三个字节：0xE4, 0xB8, 0x8D

输入与输出：文件编码与程序内码

- 词法分析器的输入是源代码文件，输出是 Token 序列
- 关键问题：非 ASCII 字符（如中文、日文、韩文）在文件存储与程序处理有一定区别

文件编码 (File Encoding)

- 源代码文件在磁盘上存储时使用的编码
- 常见编码：ASCII, UTF-8, GBK, ISO-8859-1
- 示例：中文“中”在 UTF-8 编码下存储为三个字节：0xE4, 0xB8, 0x8D

程序内码 (Internal Encoding)

- 程序在内存中处理字符时使用的编码
- 通常是 Unicode (UTF-32, UTF-16) 或 UTF-8
- 示例：“中”字符被表示为 Unicode 码点 U+4E2D

扩展知识：Unicode 编码方案详解

Unicode 基础

- 字符编码标准，为全球语言提供统一编码
- 核心目标：唯一性、通用性、兼容性
- 1991 年 Unicode 1.0 版本发布，包含 7,161 个字符
- 2023 年 Unicode 15.1 已收录 154,955 个字符

编码方案对比

- **UTF-8**：可变长度 (1-4 字节)，兼容 ASCII
- **UTF-16**：可变长度 (2 或 4 字节)，Windows 系统常用
- **UTF-32**：固定长度 (4 字节)，内存处理高效

扩展知识：Unicode 编码方案详解

Unicode 基础

- 字符编码标准，为全球语言提供统一编码
- 核心目标：唯一性、通用性、兼容性
- 1991 年 Unicode 1.0 版本发布，包含 7,161 个字符
- 2023 年 Unicode 15.1 已收录 154,955 个字符

编码方案对比

- **UTF-8**：可变长度 (1-4 字节)，兼容 ASCII
- **UTF-16**：可变长度 (2 或 4 字节)，Windows 系统常用
- **UTF-32**：固定长度 (4 字节)，内存处理高效

编码特点对比

特性	UTF-8	UTF-16	UTF-32
编码长度	可变 (1-4 字节)	可变 (2 或 4 字节)	固定 (4 字节)
ASCII 兼容	是	否	否
空间效率	高	中	低
适用场景	网络传输 文件存储	内存操作	随机访问

扩展知识：Unicode 编码方案详解

Unicode 基础

- 字符编码标准，为全球语言提供统一编码
- 核心目标：唯一性、通用性、兼容性
- 1991 年 Unicode 1.0 版本发布，包含 7,161 个字符
- 2023 年 Unicode 15.1 已收录 154,955 个字符

编码方案对比

- **UTF-8**: 可变长度 (1-4 字节), 兼容 ASCII
- **UTF-16**: 可变长度 (2 或 4 字节), Windows 系统常用
- **UTF-32**: 固定长度 (4 字节), 内存处理高效

编码特点对比

特性	UTF-8	UTF-16	UTF-32
编码长度	可变 (1-4 字节)	可变 (2 或 4 字节)	固定 (4 字节)
ASCII 兼容	是	否	否
空间效率	高	中	低
适用场景	网络传输 文件存储	内存操作	随机访问

典型应用场景

- UTF-8: 网页、JSON、XML、Markdown
- UTF-16: Windows 系统、Java、C# 字符串处理
- UTF-32: 需要快速随机访问的文本编辑

编码转换示例

字符	UTF-8 编码	程序内码	存储字节
“中”	0xE4, 0xB8, 0x8D	U+4E2D	E4 B8 8D
“a”	0x61	U+0061	61
“π”	0xC3, 0xB0	U+03C0	C3 B0

常见问题

- 文件编码与程序内码不匹配：导致乱码
 - 用 GBK 编码保存的中文文件，用 UTF-8 打开会显示乱码
- 词法分析器需要知道源文件的编码才能正确解析

词法分析器的编码处理

- 输入处理步骤：



词法分析器的编码处理

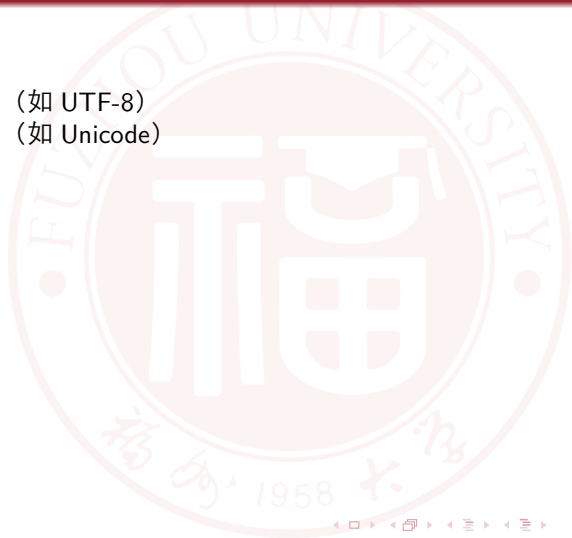
- 输入处理步骤：
 - ① 读取源文件时，指定文件编码（如 UTF-8）



词法分析器的编码处理

- 输入处理步骤：

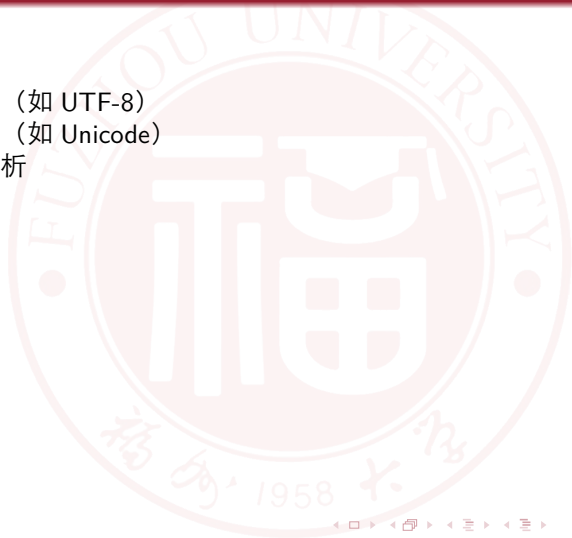
- ① 读取源文件时，指定文件编码（如 UTF-8）
- ② 将文件字节流转换为程序内码（如 Unicode）



词法分析器的编码处理

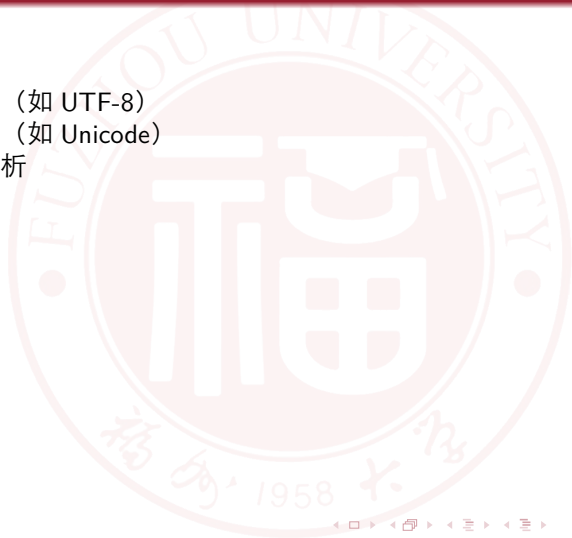
- 输入处理步骤：

- ① 读取源文件时，指定文件编码（如 UTF-8）
- ② 将文件字节流转换为程序内码（如 Unicode）
- ③ 基于转换后的字符进行词法分析



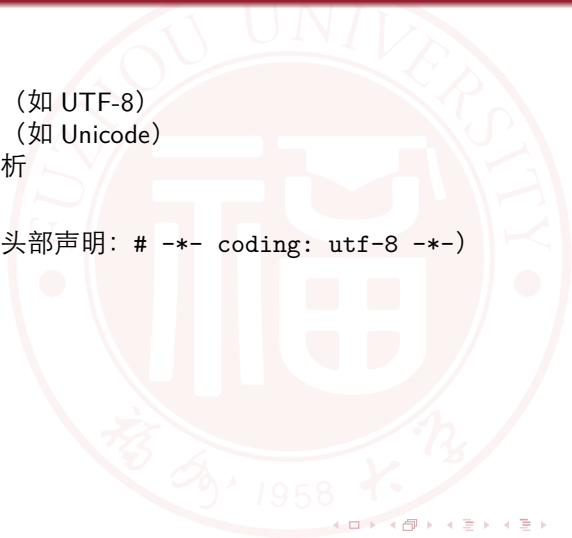
词法分析器的编码处理

- 输入处理步骤：
 - ① 读取源文件时，指定文件编码（如 UTF-8）
 - ② 将文件字节流转换为程序内码（如 Unicode）
 - ③ 基于转换后的字符进行词法分析
- 实际实现建议：



词法分析器的编码处理

- 输入处理步骤：
 - ① 读取源文件时，指定文件编码（如 UTF-8）
 - ② 将文件字节流转换为程序内码（如 Unicode）
 - ③ 基于转换后的字符进行词法分析
- 实际实现建议：
 - 明确指定文件编码（如在文件头部声明：`# -*- coding: utf-8 -*-`）



词法分析器的编码处理

- 输入处理步骤：

- ① 读取源文件时，指定文件编码（如 UTF-8）
- ② 将文件字节流转换为程序内码（如 Unicode）
- ③ 基于转换后的字符进行词法分析

- 实际实现建议：

- 明确指定文件编码（如在文件头部声明：`# -*- coding: utf-8 -*-`）
- 使用标准库的编码转换函数

词法分析器的编码处理

- 输入处理步骤：
 - ① 读取源文件时，指定文件编码（如 UTF-8）
 - ② 将文件字节流转换为程序内码（如 Unicode）
 - ③ 基于转换后的字符进行词法分析
- 实际实现建议：
 - 明确指定文件编码（如在文件头部声明：`# -*- coding: utf-8 -*-`）
 - 使用标准库的编码转换函数
 - 在 Python 中使用 `open('file.txt', 'r', encoding='utf-8')`

词法分析器的编码处理

- 输入处理步骤：
 - ① 读取源文件时，指定文件编码（如 UTF-8）
 - ② 将文件字节流转换为程序内码（如 Unicode）
 - ③ 基于转换后的字符进行词法分析
- 实际实现建议：
 - 明确指定文件编码（如在文件头部声明：`# -*- coding: utf-8 -*-`）
 - 使用标准库的编码转换函数
 - 在 Python 中使用 `open('file.txt', 'r', encoding='utf-8')`
 - 在 C/C++ 中使用 `fopen` 和 `iconv` 进行转换

词法分析器的编码处理

- 输入处理步骤：
 - ① 读取源文件时，指定文件编码（如 UTF-8）
 - ② 将文件字节流转换为程序内码（如 Unicode）
 - ③ 基于转换后的字符进行词法分析
- 实际实现建议：
 - 明确指定文件编码（如在文件头部声明：`# -*- coding: utf-8 -*-`）
 - 使用标准库的编码转换函数
 - 在 Python 中使用 `open('file.txt', 'r', encoding='utf-8')`
 - 在 C/C++ 中使用 `fopen` 和 `iconv` 进行转换
- 为什么重要：

词法分析器的编码处理

- 输入处理步骤：
 - ① 读取源文件时，指定文件编码（如 UTF-8）
 - ② 将文件字节流转换为程序内码（如 Unicode）
 - ③ 基于转换后的字符进行词法分析
- 实际实现建议：
 - 明确指定文件编码（如在文件头部声明：`# -*- coding: utf-8 -*-`）
 - 使用标准库的编码转换函数
 - 在 Python 中使用 `open('file.txt', 'r', encoding='utf-8')`
 - 在 C/C++ 中使用 `fopen` 和 `iconv` 进行转换
- 为什么重要：
 - 词法分析器需要正确识别非 ASCII 字符

词法分析器的编码处理

- 输入处理步骤：
 - ① 读取源文件时，指定文件编码（如 UTF-8）
 - ② 将文件字节流转换为程序内码（如 Unicode）
 - ③ 基于转换后的字符进行词法分析
- 实际实现建议：
 - 明确指定文件编码（如在文件头部声明：`# -*- coding: utf-8 -*-`）
 - 使用标准库的编码转换函数
 - 在 Python 中使用 `open('file.txt', 'r', encoding='utf-8')`
 - 在 C/C++ 中使用 `fopen` 和 `iconv` 进行转换
- 为什么重要：
 - 词法分析器需要正确识别非 ASCII 字符
 - 例如：中文标识符“变量”与英文标识符“variable”是不同的

词法分析器的编码处理

- 输入处理步骤：
 - ① 读取源文件时，指定文件编码（如 UTF-8）
 - ② 将文件字节流转换为程序内码（如 Unicode）
 - ③ 基于转换后的字符进行词法分析
- 实际实现建议：
 - 明确指定文件编码（如在文件头部声明：`# -*- coding: utf-8 -*-`）
 - 使用标准库的编码转换函数
 - 在 Python 中使用 `open('file.txt', 'r', encoding='utf-8')`
 - 在 C/C++ 中使用 `fopen` 和 `iconv` 进行转换
- 为什么重要：
 - 词法分析器需要正确识别非 ASCII 字符
 - 例如：中文标识符“变量”与英文标识符“variable”是不同的
 - 错误的编码处理会导致词法错误或识别错误

目录

① 知识点回顾

② 词法分析

③ 手写一个词法分析器

输入与输出

何时分割单词

单词的类别

词法分析程序的状态与转移

④ 小结与作业



基本规则

- 词法分析器需要确定字符流中单词的边界 (token 边界)



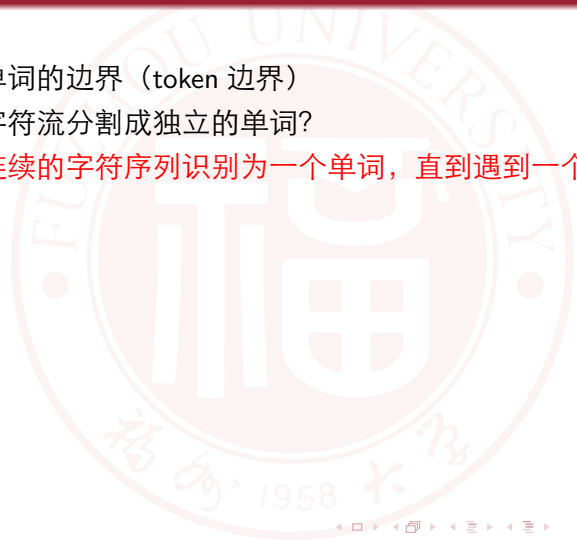
基本规则

- 词法分析器需要确定字符流中单词的边界（token 边界）
- 关键问题：在哪些位置应该将字符流分割成独立的单词？



基本规则

- 词法分析器需要确定字符流中单词的边界（token 边界）
- 关键问题：在哪些位置应该将字符流分割成独立的单词？
- 基本原则：根据词法规则，将连续的字符序列识别为一个单词，直到遇到一个分割点



基本规则

- 词法分析器需要确定字符流中单词的边界（token 边界）
- 关键问题：在哪些位置应该将字符流分割成独立的单词？
- 基本原则：根据词法规则，将连续的字符序列识别为一个单词，直到遇到一个分割点

分割点的常见类型

基本规则

- 词法分析器需要确定字符流中单词的边界（token 边界）
- 关键问题：在哪些位置应该将字符流分割成独立的单词？
- 基本原则：根据词法规则，将连续的字符序列识别为一个单词，直到遇到一个分割点

分割点的常见类型

- 空白字符：空格、制表符、换行符（通常作为单词之间的分隔符）

基本规则

- 词法分析器需要确定字符流中单词的边界（token 边界）
- 关键问题：在哪些位置应该将字符流分割成独立的单词？
- 基本原则：根据词法规则，将连续的字符序列识别为一个单词，直到遇到一个分割点

分割点的常见类型

- 空白字符：空格、制表符、换行符（通常作为单词之间的分隔符）
- 运算符： $+$ $-$ $*$ $/$ $=$ $<$ $>$ 等（通常自身就是一个单词）

基本规则

- 词法分析器需要确定字符流中单词的边界（token 边界）
- 关键问题：在哪些位置应该将字符流分割成独立的单词？
- 基本原则：根据词法规则，将连续的字符序列识别为一个单词，直到遇到一个分割点

分割点的常见类型

- 空白字符：空格、制表符、换行符（通常作为单词之间的分隔符）
- 运算符：+ - * / = < >等（通常自身就是一个单词）
- 界符：（ ） [] ; , 等（通常自身就是一个单词）

基本规则

- 词法分析器需要确定字符流中单词的边界（token 边界）
- 关键问题：在哪些位置应该将字符流分割成独立的单词？
- 基本原则：根据词法规则，将连续的字符序列识别为一个单词，直到遇到一个分割点

分割点的常见类型

- 空白字符：空格、制表符、换行符（通常作为单词之间的分隔符）
- 运算符：+ - * / = < >等（通常自身就是一个单词）
- 界符：（ ） [] ; , 等（通常自身就是一个单词）
- **非词法字符：不被语言接受的字符（如非法字符，会触发错误）**

示例：C 语言代码

```
int a = 10 + b * c;
```

示例：C 语言代码

```
int a = 10 + b * c;
```

- 分割点：空格、=、+、*、;

示例：C 语言代码

```
int a = 10 + b * c;
```

- 分割点：空格、=、+、*、;
- 单词：int, a, =, 10, +, b, *, c, ;

分割规则的细节

空白字符处理

- 空格、`\{nrt}`通常作为单词分隔符



分割规则的细节

空白字符处理

- 空格、`\{nrt}`通常作为单词分隔符
- 某些语言用换行符表示语句结束



分割规则的细节

空白字符处理

- 空格、`\{nrt}`通常作为单词分隔符
- 某些语言用换行符表示语句结束
- 词法分析器通常会跳过空白字符，不产生 Token



分割规则的细节

空白字符处理

- 空格、`\{nrt}`通常作为单词分隔符
- 某些语言用换行符表示语句结束
- 词法分析器通常会跳过空白字符，不产生 Token
- 例外：字符串常量中的空白字符保留

分割规则的细节

空白字符处理

- 空格、`\{nrt}`通常作为单词分隔符
- 某些语言用换行符表示语句结束
- 词法分析器通常会跳过空白字符，不产生 Token
- 例外：字符串常量中的空白字符保留

运算符与界符

分割规则的细节

空白字符处理

- 空格、`\{nrt}`通常作为单词分隔符
- 某些语言用换行符表示语句结束
- 词法分析器通常会跳过空白字符，不产生 Token
- 例外：字符串常量中的空白字符保留

运算符与界符

- 单个字符：`+`, `-`, `*`, `/`, `=`, etc.

分割规则的细节

空白字符处理

- 空格、`\{nrt}`通常作为单词分隔符
- 某些语言用换行符表示语句结束
- 词法分析器通常会跳过空白字符，不产生 Token
- 例外：字符串常量中的空白字符保留

运算符与界符

- 单个字符：`+`, `-`, `*`, `/`, `=`, etc.
- 多字符：`==`, `!=`, `<=`, `>=`, etc.

分割规则的细节

空白字符处理

- 空格、`\{nrt}`通常作为单词分隔符
- 某些语言用换行符表示语句结束
- 词法分析器通常会跳过空白字符，不产生 Token
- 例外：字符串常量中的空白字符保留

运算符与界符

- 单个字符：`+`, `-`, `*`, `/`, `=`, etc.
- 多字符：`==`, `!=`, `<=`, `>=`, etc.
- 界符：`( )` `[ ]` ; , ...

分割规则的细节

空白字符处理

- 空格、`\{nrt}`通常作为单词分隔符
- 某些语言用换行符表示语句结束
- 词法分析器通常会跳过空白字符，不产生 Token
- 例外：字符串常量中的空白字符保留

运算符与界符

- 单个字符：`+`, `-`, `*`, `/`, `=`, etc.
- 多字符：`==`, `!=`, `<=`, `>=`, etc.
- 界符：`( )` `[ ]` `;` `,` `...`

标识符与关键字

分割规则的细节

空白字符处理

- 空格、`\{nrt}`通常作为单词分隔符
- 某些语言用换行符表示语句结束
- 词法分析器通常会跳过空白字符，不产生 Token
- 例外：字符串常量中的空白字符保留

运算符与界符

- 单个字符：`+`, `-`, `*`, `/`, `=`, etc.
- 多字符：`==`, `!=`, `<=`, `>=`, etc.
- 界符：`( )` `[ ]` `;` `,` `...`

标识符与关键字

- 标识符：字母开头，后跟字母或数字

分割规则的细节

空白字符处理

- 空格、`\{nrt}`通常作为单词分隔符
- 某些语言用换行符表示语句结束
- 词法分析器通常会跳过空白字符，不产生 Token
- 例外：字符串常量中的空白字符保留

运算符与界符

- 单个字符：`+`, `-`, `*`, `/`, `=`, etc.
- 多字符：`==`, `!=`, `<=`, `>=`, etc.
- 界符：`( )` `[ ]` `;` `,` `...`

标识符与关键字

- 标识符：字母开头，后跟字母或数字
- 关键字：保留字（如 `int`, `if`, `while`），不能用作标识符

分割规则的细节

空白字符处理

- 空格、`\{nrt}`通常作为单词分隔符
- 某些语言用换行符表示语句结束
- 词法分析器通常会跳过空白字符，不产生 Token
- 例外：字符串常量中的空白字符保留

运算符与界符

- 单个字符：`+`, `-`, `*`, `/`, `=`, etc.
- 多字符：`==`, `!=`, `<=`, `>=`, etc.
- 界符：`( )` `[ ]` `;` `,` `...`

标识符与关键字

- 标识符：字母开头，后跟字母或数字
- 关键字：保留字（如 `int`, `if`, `while`），不能用作标识符
- **关键规则：关键字优先于标识符**

分割规则的细节

空白字符处理

- 空格、`\{nrt}`通常作为单词分隔符
- 某些语言用换行符表示语句结束
- 词法分析器通常会跳过空白字符，不产生 Token
- 例外：字符串常量中的空白字符保留

运算符与界符

- 单个字符：`+`, `-`, `*`, `/`, `=`, etc.
- 多字符：`==`, `!=`, `<=`, `>=`, etc.
- 界符：`( )` `[ ]` `;` `,` `...`

标识符与关键字

- 标识符：字母开头，后跟字母或数字
- 关键字：保留字（如 `int`, `if`, `while`），不能用作标识符
- 关键规则：关键字优先于标识符

数字常量

分割规则的细节

空白字符处理

- 空格、`\{nrt}`通常作为单词分隔符
- 某些语言用换行符表示语句结束
- 词法分析器通常会跳过空白字符，不产生 Token
- 例外：字符串常量中的空白字符保留

运算符与界符

- 单个字符：`+`, `-`, `*`, `/`, `=`, etc.
- 多字符：`==`, `!=`, `<=`, `>=`, etc.
- 界符：`( )` `[ ]` `;` `,` `...`

标识符与关键字

- 标识符：字母开头，后跟字母或数字
- 关键字：保留字（如 `int`, `if`, `while`），不能用作标识符
- 关键规则：关键字优先于标识符

数字常量

- 整数：数字序列（可能包含前导 0）

分割规则的细节

空白字符处理

- 空格、`\{nrt}`通常作为单词分隔符
- 某些语言用换行符表示语句结束
- 词法分析器通常会跳过空白字符，不产生 Token
- 例外：字符串常量中的空白字符保留

运算符与界符

- 单个字符：`+`, `-`, `*`, `/`, `=`, etc.
- 多字符：`==`, `!=`, `<=`, `>=`, etc.
- 界符：`( )` `[ ]` `;` `,` `...`

标识符与关键字

- 标识符：字母开头，后跟字母或数字
- 关键字：保留字（如 `int`, `if`, `while`），不能用作标识符
- 关键规则：关键字优先于标识符

数字常量

- 整数：数字序列（可能包含前导 0）
- 浮点数：整数、小数点、小数

分割规则的细节

空白字符处理

- 空格、`\{nrt}`通常作为单词分隔符
- 某些语言用换行符表示语句结束
- 词法分析器通常会跳过空白字符，不产生 Token
- 例外：字符串常量中的空白字符保留

运算符与界符

- 单个字符：`+`, `-`, `*`, `/`, `=`, etc.
- 多字符：`==`, `!=`, `<=`, `>=`, etc.
- 界符：`( )` `[ ]` `;` `,` `...`

标识符与关键字

- 标识符：字母开头，后跟字母或数字
- 关键字：保留字（如 `int`, `if`, `while`），不能用作标识符
- 关键规则：关键字优先于标识符

数字常量

- 整数：数字序列（可能包含前导 0）
- 浮点数：整数、小数点、小数
- 规则：直到遇到非数字字符为止???

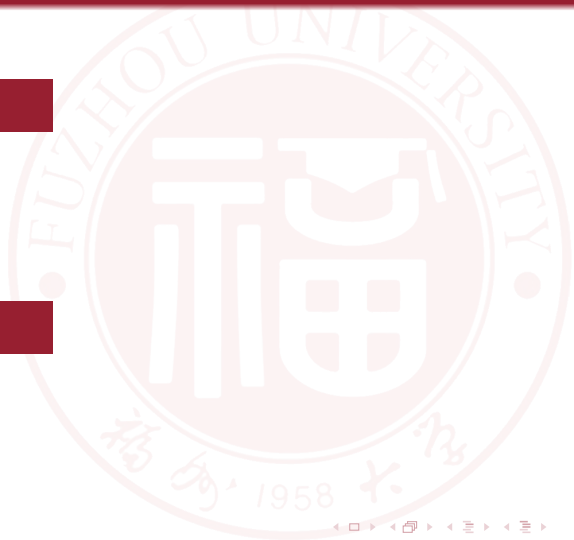
特殊情况处理

连续运算符

- ++：自增运算符（不是两个+）
- --：自减运算符（不是两个-）
- +=：赋值运算符（不是+和=）

数字与标识符

- 123abc：不是一般数字常量
- abc123：标识符



特殊情况处理

连续运算符

- ++：自增运算符（不是两个+）
- --：自减运算符（不是两个-）
- +=：赋值运算符（不是+和=）

数字与标识符

- 123abc：不是一般数字常量
- abc123：标识符

字符串常量

- 以双引号" 或单引号' 开始和结束
- 内部的空白字符保留
- 转义字符（如\n）被处理为单个字符
- 分割点：遇到结束引号

特殊情况处理

连续运算符

- ++：自增运算符（不是两个+）
- --：自减运算符（不是两个-）
- +=：赋值运算符（不是+和=）

数字与标识符

- 123abc：不是一般数字常量
- abc123：标识符

字符串常量

- 以双引号" 或单引号' 开始和结束
- 内部的空白字符保留
- 转义字符（如\n）被处理为单个字符
- 分割点：遇到结束引号

注释

- C 语言：/* ... */和// ...
- 注意：注释被跳过，不产生 Token

目录

① 知识点回顾

② 词法分析

③ 手写一个词法分析器

输入与输出

何时分割单词

单词的类别

词法分析程序的状态与转移

④ 小结与作业



单词的类别：基本分类

- 词法分析器将字符流分割成有意义的单词 (token)，每个单词属于特定的类别



单词的类别：基本分类

- 词法分析器将字符流分割成有意义的单词 (token)，每个单词属于特定的类别
- 核心概念：单词类别决定了词法分析器如何处理该单词**



单词的类别：基本分类

- 词法分析器将字符流分割成有意义的单词 (token)，每个单词属于特定的类别
- 核心概念：单词类别决定了词法分析器如何处理该单词
- 常见类别如下（按语言特性）：

单词的类别：基本分类

- 词法分析器将字符流分割成有意义的单词 (token)，每个单词属于特定的类别
- 核心概念：单词类别决定了词法分析器如何处理该单词
- 常见类别如下（按语言特性）：

基础类别

单词的类别：基本分类

- 词法分析器将字符流分割成有意义的单词 (token)，每个单词属于特定的类别
- 核心概念：单词类别决定了词法分析器如何处理该单词
- 常见类别如下（按语言特性）：

基础类别

- 关键字：语言保留字

单词的类别：基本分类

- 词法分析器将字符流分割成有意义的单词 (token)，每个单词属于特定的类别
- 核心概念：单词类别决定了词法分析器如何处理该单词
- 常见类别如下（按语言特性）：

基础类别

- 关键字：语言保留字
- 标识符：用户定义的名称（变量、函数等）

单词的类别：基本分类

- 词法分析器将字符流分割成有意义的单词 (token)，每个单词属于特定的类别
- 核心概念：单词类别决定了词法分析器如何处理该单词
- 常见类别如下（按语言特性）：

基础类别

- 关键字：语言保留字
- 标识符：用户定义的名称（变量、函数等）
- 运算符：+、-、*、/、=等

单词的类别：基本分类

- 词法分析器将字符流分割成有意义的单词 (token)，每个单词属于特定的类别
- 核心概念：单词类别决定了词法分析器如何处理该单词
- 常见类别如下（按语言特性）：

基础类别

- 关键字：语言保留字
- 标识符：用户定义的名称（变量、函数等）
- 运算符：+、-、*、/、=等
- 界符：;、\、(、)等

单词的类别：基本分类

- 词法分析器将字符流分割成有意义的单词 (token)，每个单词属于特定的类别
- 核心概念：单词类别决定了词法分析器如何处理该单词
- 常见类别如下（按语言特性）：

基础类别

- 关键字：语言保留字
- 标识符：用户定义的名称（变量、函数等）
- 运算符：+、-、*、/、=等
- 界符：;、\、(、)等

常量与特殊类别

单词的类别：基本分类

- 词法分析器将字符流分割成有意义的单词（token），每个单词属于特定的类别
- 核心概念：单词类别决定了词法分析器如何处理该单词
- 常见类别如下（按语言特性）：

基础类别

- 关键字：语言保留字
- 标识符：用户定义的名称（变量、函数等）
- 运算符：+、-、*、/、=等
- 界符：;、\、(、)等

常量与特殊类别

- 常量：固定值（整数、浮点数、字符串等）

单词的类别：基本分类

- 词法分析器将字符流分割成有意义的单词（token），每个单词属于特定的类别
- 核心概念：单词类别决定了词法分析器如何处理该单词
- 常见类别如下（按语言特性）：

基础类别

- 关键字：语言保留字
- 标识符：用户定义的名称（变量、函数等）
- 运算符：+、-、*、/、=等
- 界符：;、\、(、)等

常量与特殊类别

- 常量：固定值（整数、浮点数、字符串等）
- 注释：源代码中的说明文字（通常被忽略）

单词的类别：基本分类

- 词法分析器将字符流分割成有意义的单词 (token)，每个单词属于特定的类别
- 核心概念：单词类别决定了词法分析器如何处理该单词
- 常见类别如下（按语言特性）：

基础类别

- 关键字：语言保留字
- 标识符：用户定义的名称（变量、函数等）
- 运算符：+、-、*、/、=等
- 界符：;、,、(、)等

常量与特殊类别

- 常量：固定值（整数、浮点数、字符串等）
- 注释：源代码中的说明文字（通常被忽略）
- 错误：无效的字符序列（词法错误）

单词分类的重要性

- 语法分析依赖：语法分析器根据单词类别构建语法树
- 语义分析基础：类别决定了后续分析的处理方式
- 错误检测：类别错误（如关键字用作标识符）是常见词法错误

关键字与标识符：关键区别

关键字 (Keywords)

- 语言保留字，不能用作标识符



关键字与标识符：关键区别

关键字 (Keywords)

- 语言保留字，不能用作标识符
- 常见关键字：if, else, while, for, int, char, return



关键字与标识符：关键区别

关键字 (Keywords)

- 语言保留字，不能用作标识符
- 常见关键字：if, else, while, for, int, char, return
- 处理规则：直接识别为关键字类别，不进入标识符处理



关键字与标识符：关键区别

关键字 (Keywords)

- 语言保留字，不能用作标识符
- 常见关键字：if, else, while, for, int, char, return
- 处理规则：直接识别为关键字类别，不进入标识符处理
- 示例：if 是关键字，if1 是标识符

关键字与标识符：关键区别

关键字 (Keywords)

- 语言保留字，不能用作标识符
- 常见关键字：if, else, while, for, int, char, return
- 处理规则：直接识别为关键字类别，不进入标识符处理
- 示例：if 是关键字，if1 是标识符

标识符 (Identifiers)

关键字与标识符：关键区别

关键字 (Keywords)

- 语言保留字，不能用作标识符
- 常见关键字：if, else, while, for, int, char, return
- 处理规则：直接识别为关键字类别，不进入标识符处理
- 示例：if 是关键字，if1 是标识符

标识符 (Identifiers)

- 用户定义的名称，用于表示变量、函数等

关键字与标识符：关键区别

关键字 (Keywords)

- 语言保留字，不能用作标识符
- 常见关键字：if, else, while, for, int, char, return
- 处理规则：直接识别为关键字类别，不进入标识符处理
- 示例：if 是关键字，if1 是标识符

标识符 (Identifiers)

- 用户定义的名称，用于表示变量、函数等
- **命名规则：以字母或下划线开头，后跟字母、数字或下划线**

关键字与标识符：关键区别

关键字 (Keywords)

- 语言保留字，不能用作标识符
- 常见关键字：if, else, while, for, int, char, return
- 处理规则：直接识别为关键字类别，不进入标识符处理
- 示例：if 是关键字，if1 是标识符

标识符 (Identifiers)

- 用户定义的名称，用于表示变量、函数等
- 命名规则：以字母或下划线开头，后跟字母、数字或下划线
- 大小写敏感：大多数语言中 Var 和 var 不同

关键字与标识符：关键区别

关键字 (Keywords)

- 语言保留字，不能用作标识符
- 常见关键字：if, else, while, for, int, char, return
- 处理规则：直接识别为关键字类别，不进入标识符处理
- 示例：if 是关键字，if1 是标识符

标识符 (Identifiers)

- 用户定义的名称，用于表示变量、函数等
- 命名规则：以字母或下划线开头，后跟字母、数字或下划线
- 大小写敏感：大多数语言中 Var 和 var 不同
- 大小写不敏感：Basic、SQL 等语言中大小写是不敏感的

关键字与标识符：关键区别

关键字 (Keywords)

- 语言保留字，不能用作标识符
- 常见关键字：if, else, while, for, int, char, return
- 处理规则：直接识别为关键字类别，不进入标识符处理
- 示例：if 是关键字，if1 是标识符

标识符 (Identifiers)

- 用户定义的名称，用于表示变量、函数等
- 命名规则：以字母或下划线开头，后跟字母、数字或下划线
- 大小写敏感：大多数语言中 Var 和 var 不同
- 大小写不敏感：Basic、SQL 等语言中大小写是不敏感的
- 示例：count, sum_value, _temp

运算符与界符：常见类型

运算符 (Operators)

- 一元运算符：+、-、!、~



运算符与界符：常见类型

运算符 (Operators)

- 一元运算符：+、-、!、~
- 二元运算符：+、-、*、/、=、<



运算符与界符：常见类型

运算符 (Operators)

- 一元运算符：+、-、!、~
- 二元运算符：+、-、*、/、=、<
- 多字符运算符：==、!=、<=、>=、++、--



运算符与界符：常见类型

运算符 (Operators)

- 一元运算符：+、-、!、~
- 二元运算符：+、-、*、/、=、<
- 多字符运算符：==、!=、<=、>=、++、--
- 运算符优先级：在语法分析中很重要



运算符与界符：常见类型

运算符 (Operators)

- 一元运算符：+、-、!、~
- 二元运算符：+、-、*、/、=、<
- 多字符运算符：==、!=、<=、>=、++、--
- 运算符优先级：在语法分析中很重要

界符 (Delimiters)

运算符与界符：常见类型

运算符 (Operators)

- 一元运算符：+、-、!、~
- 二元运算符：+、-、*、/、=、<
- 多字符运算符：==、!=、<=、>=、++、--
- 运算符优先级：在语法分析中很重要

界符 (Delimiters)

- 分隔符：,、;、.

运算符与界符：常见类型

运算符 (Operators)

- 一元运算符：+、-、!、~
- 二元运算符：+、-、*、/、=、<
- 多字符运算符：==、!=、<=、>=、++、--
- 运算符优先级：在语法分析中很重要

界符 (Delimiters)

- 分隔符：,、;、.
- 分组符：(、)、{、}、[、]

运算符与界符：常见类型

运算符 (Operators)

- 一元运算符：+、-、!、~
- 二元运算符：+、-、*、/、=、<
- 多字符运算符：==、!=、<=、>=、++、--
- 运算符优先级：在语法分析中很重要

界符 (Delimiters)

- 分隔符：,、;、.
- 分组符：(、)、{、}、[、]
- 特殊符号：::、-> (在 C++ 等语言中)

运算符与界符：常见类型

运算符 (Operators)

- 一元运算符：+、-、!、~
- 二元运算符：+、-、*、/、=、<
- 多字符运算符：==、!=、<=、>=、++、--
- 运算符优先级：在语法分析中很重要

界符 (Delimiters)

- 分隔符：,、;、.
- 分组符：(、)、{、}、[、]
- 特殊符号：::、->（在 C++ 等语言中）
- 语句结束符：;（C、Java 等语言）

运算符与界符：常见类型

运算符 (Operators)

- 一元运算符：+、-、!、~
- 二元运算符：+、-、*、/、=、<
- 多字符运算符：==、!=、<=、>=、++、--
- 运算符优先级：在语法分析中很重要

界符 (Delimiters)

- 分隔符：,、;、.
- 分组符：(、)、{、}、[、]
- 特殊符号：::、->（在 C++ 等语言中）
- 语句结束符：;（C、Java 等语言）

运算符识别规则

- 词法分析器需要识别最长匹配的运算符
- 例如：==是单个运算符，不是=和=
- > =是两个独立的运算符（>和=）

常量类别：常见类型

整数常量 (Integer Literals)

- 十进制：123、0、-456
- 八进制：0123（以0开头）
- 十六进制：0x1A、0xFF
- 规则：必须以数字开头，不能包含非数字字符

浮点数常量 (Floating-Point Literals)

- 基本形式：3.14、0.5、2.0e-3
- 科学计数法：1.23e+4、6.02e23
- 规则：必须包含小数点或指数部分

常量类别：常见类型

整数常量 (Integer Literals)

- 十进制：123、0、-456
- 八进制：0123（以0开头）
- 十六进制：0x1A、0xFF
- 规则：必须以数字开头，不能包含非数字字符

字符串常量 (String Literals)

- 双引号："Hello, world!"
- 单引号：'a'（字符常量）
- 转义字符：\n、\t、\"

浮点常量 (Floating-Point Literals)

- 基本形式：3.14、0.5、2.0e-3
- 科学计数法：1.23e+4、6.02e23
- 规则：必须包含小数点或指数部分

常量类别：常见类型

整数常量 (Integer Literals)

- 十进制：123、0、-456
- 八进制：0123（以0开头）
- 十六进制：0x1A、0xFF
- 规则：必须以数字开头，不能包含非数字字符

字符串常量 (String Literals)

- 双引号："Hello, world!"
- 单引号：'a'（字符常量）
- 转义字符：\n、\t、\"

浮点常量 (Floating-Point Literals)

- 基本形式：3.14、0.5、2.0e-3
- 科学计数法：1.23e+4、6.02e23
- 规则：必须包含小数点或指数部分

布尔常量 (Boolean Literals)

- true、false
- 规则：通常小写，不带引号

目录

① 知识点回顾

② 词法分析

③ 手写一个词法分析器

输入与输出

何时分割单词

单词的类别

词法分析程序的状态与转移

④ 小结与作业



状态与转移：程序逻辑视角

- 词法分析器在程序中通过状态变量跟踪当前分析位置



状态与转移：程序逻辑视角

- 词法分析器在程序中通过状态变量跟踪当前分析位置
- 关键概念：状态表示当前正在识别什么类型的单词**



状态与转移：程序逻辑视角

- 词法分析器在程序中通过状态变量跟踪当前分析位置
- 关键概念：状态表示当前正在识别什么类型的单词
- 核心逻辑：根据当前字符和状态，决定下一个状态



状态与转移：程序逻辑视角

- 词法分析器在程序中通过状态变量跟踪当前分析位置
- 关键概念：状态表示当前正在识别什么类型的单词
- 核心逻辑：根据当前字符和状态，决定下一个状态
- 状态转移：从一个状态到另一个状态的转换过程**

状态与转移：程序逻辑视角

- 词法分析器在程序中通过状态变量跟踪当前分析位置
- 关键概念：状态表示当前正在识别什么类型的单词
- 核心逻辑：根据当前字符和状态，决定下一个状态
- 状态转移：从一个状态到另一个状态的转换过程

状态表示

- 状态可以是整数、枚举或字符串
- 常见状态：初始状态、注释状态、标识符状态、数字状态、运算符状态等
- 示例：C 语言词法分析器可能有以下状态
 - STATE_START：初始状态
 - STATE_IDENTIFIER：正在识别标识符
 - STATE_NUMBER：正在识别数字
 - STATE_OPERATOR：正在识别运算符

状态转移：C 语言风格

状态转移逻辑

- 词法分析器使用状态变量跟踪当前分析状态

状态转移：C 语言风格

状态转移逻辑

- 词法分析器使用状态变量跟踪当前分析状态
- 根据当前字符和状态，决定如何处理

状态转移：C 语言风格

状态转移逻辑

- 词法分析器使用状态变量跟踪当前分析状态
- 根据当前字符和状态，决定如何处理
- 状态转移是词法分析的核心逻辑

状态转移代码示例

```

1 // 词法分析器状态变量
2 enum TokenState { START, IDENT, NUMBER, OPERATOR, STRING };
3 TokenState state = START;
4 std::string current_token;
5 // 读取字符并处理
6 while (char c = get_next_char()) {
7     switch(state) {
8         case START:
9             ...
10        case IDENT:
11            ...
12        case NUMBER:
13            ...
14    }
15 }
    
```

状态转移的关键点

核心原则

- 最长匹配：始终尝试识别最长的可能单词



状态转移的关键点

核心原则

- 最长匹配：始终尝试识别最长的可能单词
- 状态保持：在识别过程中保持在当前状态

状态转移的关键点

核心原则

- 最长匹配：始终尝试识别最长的可能单词
- 状态保持：在识别过程中保持在当前状态
- 状态重置：识别完成或遇到分割点时重置状态

状态转移的关键点

核心原则

- 最长匹配：始终尝试识别最长的可能单词
- 状态保持：在识别过程中保持在当前状态
- 状态重置：识别完成或遇到分割点时重置状态

示例：运算符

- 输入：==

状态转移的关键点

核心原则

- 最长匹配：始终尝试识别最长的可能单词
- 状态保持：在识别过程中保持在当前状态
- 状态重置：识别完成或遇到分割点时重置状态

示例：运算符

- 输入：==
- 状态转移：

状态转移的关键点

核心原则

- 最长匹配：始终尝试识别最长的可能单词
- 状态保持：在识别过程中保持在当前状态
- 状态重置：识别完成或遇到分割点时重置状态

示例：运算符

- 输入：==
- 状态转移：
 - ① START 状态，遇到=，进入 OPERATOR 状态

状态转移的关键点

核心原则

- 最长匹配：始终尝试识别最长的可能单词
- 状态保持：在识别过程中保持在当前状态
- 状态重置：识别完成或遇到分割点时重置状态

示例：运算符

- 输入：==
- 状态转移：
 - ① START 状态，遇到=，进入 OPERATOR 状态
 - ② OPERATOR 状态，遇到第二个=，继续 OPERATOR 状态

状态转移的关键点

核心原则

- 最长匹配：始终尝试识别最长的可能单词
- 状态保持：在识别过程中保持在当前状态
- 状态重置：识别完成或遇到分割点时重置状态

示例：运算符

- 输入：==
- 状态转移：
 - ① START 状态，遇到=，进入 OPERATOR 状态
 - ② OPERATOR 状态，遇到第二个=，继续 OPERATOR 状态
 - ③ 遇到非运算符字符，识别为==（单个运算符）

状态转移的关键点

核心原则

- 最长匹配：始终尝试识别最长的可能单词
- 状态保持：在识别过程中保持在当前状态
- 状态重置：识别完成或遇到分割点时重置状态

示例：标识符

- 输入：variable

状态转移的关键点

核心原则

- 最长匹配：始终尝试识别最长的可能单词
- 状态保持：在识别过程中保持在当前状态
- 状态重置：识别完成或遇到分割点时重置状态

示例：标识符

- 输入：variable
- 状态转移：

状态转移的关键点

核心原则

- 最长匹配：始终尝试识别最长的可能单词
- 状态保持：在识别过程中保持在当前状态
- 状态重置：识别完成或遇到分割点时重置状态

示例：标识符

- 输入：variable
- 状态转移：
 - ① START 状态，遇到 v，进入 IDENT 状态

状态转移的关键点

核心原则

- 最长匹配：始终尝试识别最长的可能单词
- 状态保持：在识别过程中保持在当前状态
- 状态重置：识别完成或遇到分割点时重置状态

示例：标识符

- 输入：variable
- 状态转移：
 - ① START 状态，遇到 v，进入 IDENT 状态
 - ② IDENT 状态，遇到 a、r、i、a、b、l、e，保持 IDENT 状态

状态转移的关键点

核心原则

- 最长匹配：始终尝试识别最长的可能单词
- 状态保持：在识别过程中保持在当前状态
- 状态重置：识别完成或遇到分割点时重置状态

示例：标识符

- 输入：variable
- 状态转移：
 - ① START 状态，遇到 v，进入 IDENT 状态
 - ② IDENT 状态，遇到 a、r、i、a、b、l、e，保持 IDENT 状态
 - ③ 遇到非字母数字字符，识别为 variable

状态转移的关键点

核心原则

- 最长匹配：始终尝试识别最长的可能单词
- 状态保持：在识别过程中保持在当前状态
- 状态重置：识别完成或遇到分割点时重置状态

示例：数字

- 输入：123abc

状态转移的关键点

核心原则

- 最长匹配：始终尝试识别最长的可能单词
- 状态保持：在识别过程中保持在当前状态
- 状态重置：识别完成或遇到分割点时重置状态

示例：数字

- 输入：123abc
- 状态转移：

状态转移的关键点

核心原则

- 最长匹配：始终尝试识别最长的可能单词
- 状态保持：在识别过程中保持在当前状态
- 状态重置：识别完成或遇到分割点时重置状态

示例：数字

- 输入：123abc
- 状态转移：
 - ① START 状态，遇到 1，进入 NUMBER 状态

状态转移的关键点

核心原则

- 最长匹配：始终尝试识别最长的可能单词
- 状态保持：在识别过程中保持在当前状态
- 状态重置：识别完成或遇到分割点时重置状态

示例：数字

- 输入：123abc
- 状态转移：
 - ① START 状态，遇到 1，进入 NUMBER 状态
 - ② NUMBER 状态，遇到 2、3，保持 NUMBER 状态

状态转移的关键点

核心原则

- 最长匹配：始终尝试识别最长的可能单词
- 状态保持：在识别过程中保持在当前状态
- 状态重置：识别完成或遇到分割点时重置状态

示例：数字

- 输入：123abc
- 状态转移：
 - ① START 状态，遇到 1，进入 NUMBER 状态
 - ② NUMBER 状态，遇到 2、3，保持 NUMBER 状态
 - ③ 遇到 a，识别为 123（数字常量）

状态转移的关键点

核心原则

- 最长匹配：始终尝试识别最长的可能单词
- 状态保持：在识别过程中保持在当前状态
- 状态重置：识别完成或遇到分割点时重置状态

示例：数字

- 输入：123abc
- 状态转移：
 - ① START 状态，遇到 1，进入 NUMBER 状态
 - ② NUMBER 状态，遇到 2、3，保持 NUMBER 状态
 - ③ 遇到 a，识别为 123（数字常量）
 - ④ 重置状态，从 a 开始识别标识符

状态转移的实现细节

辅助函数函数

- `is_alpha(c)`: 判断字符是否为字母
- `is_digit(c)`: 判断字符是否为数字
- `is_operator(c)`: 判断字符是否为运算符
- `emit_token(type, value)`: 输出识别到的单词
- `unget_char(c)`: 将字符放回输入流

状态转移的实现细节

辅助函数函数

- `is_alpha(c)`: 判断字符是否为字母
- `is_digit(c)`: 判断字符是否为数字
- `is_operator(c)`: 判断字符是否为运算符
- `emit_token(type, value)`: 输出识别到的单词
- `unget_char(c)`: 将字符放回输入流

常见错误

- 未处理最长匹配 (如`==`被识别为两个`=`)
- 未正确重置状态 (如数字后未重置状态)
- 未正确处理边界情况 (如数字以`+`或`-`开头)

状态转移的实现细节（续）

实现

- 优先处理关键字：在识别标识符前先检查关键字
- 使用状态表：用二维数组存储状态转移逻辑
- 错误处理：在状态转移中添加错误处理逻辑
- 字符缓冲：使用缓冲区保存已读字符

状态转移的实现细节（续）

实现

- 优先处理关键字：在识别标识符前先检查关键字
- 使用状态表：用二维数组存储状态转移逻辑
- 错误处理：在状态转移中添加错误处理逻辑
- 字符缓冲：使用缓冲区保存已读字符

调试

- 打印状态转移过程
- 使用测试用例验证状态转移
- 从简单语言开始，逐步添加复杂性

状态转移的实现细节（续 2）

关键：状态转移的顺序

- 状态转移必须按照优先级顺序进行
 - 先检查关键字，再检查标识符
 - 先检查多字符运算符，再检查单字符运算符
- 顺序错误会导致识别错误

状态转移的实现细节（续 2）

关键：状态转移的顺序

- 状态转移必须按照优先级顺序进行
 - 先检查关键字，再检查标识符
 - 先检查多字符运算符，再检查单字符运算符
- 顺序错误会导致识别错误

状态转移的核心逻辑

- 从 START 状态开始
- 根据第一个字符决定进入哪个状态
- 在每个状态中，根据后续字符决定如何转移
- 识别到单词后，重置状态并继续

目录



- ① 知识点回顾
- ② 词法分析
- ③ 手写一个词法分析器
- ④ 小结与作业

本讲小结：词法分析器核心要点

- 词法分析器的核心功能
 - 输入：源代码字符流
 - 输出：Token 序列（单词类别 + 属性）
 - 核心任务：将字符流分割成有意义的单词



本讲小结：词法分析器核心要点

- 词法分析器的核心功能
 - 输入：源代码字符流
 - 输出：Token 序列（单词类别 + 属性）
 - 核心任务：将字符流分割成有意义的单词
- 关键处理内容
 - 滤除无用成分：注释、空白字符
 - 识别并归类单词：关键字、标识符、运算符、常量
 - 管理符号表：标识符的存储与查找
 - 处理字面量：数字常量、字符串常量
 - 错误检测与报告

本讲小结：词法分析器的实现要点

状态与转移：程序逻辑

- 状态表示：START, IDENT, NUMBER, OPERATOR 等

实现关键

本讲小结：词法分析器的实现要点

状态与转移：程序逻辑

- 状态表示：START, IDENT, NUMBER, OPERATOR 等
- 状态转移：根据当前字符决定下一个状态

实现关键

本讲小结：词法分析器的实现要点

状态与转移：程序逻辑

- 状态表示：START, IDENT, NUMBER, OPERATOR 等
- 状态转移：根据当前字符决定下一个状态
- **核心原则：**

实现关键

本讲小结：词法分析器的实现要点

状态与转移：程序逻辑

- 状态表示：START, IDENT, NUMBER, OPERATOR 等
- 状态转移：根据当前字符决定下一个状态
- 核心原则：
 - 最长匹配：识别最长可能单词

实现关键

本讲小结：词法分析器的实现要点

状态与转移：程序逻辑

- 状态表示：START, IDENT, NUMBER, OPERATOR 等
- 状态转移：根据当前字符决定下一个状态
- 核心原则：
 - 最长匹配：识别最长可能单词
 - 优先级：关键字 > 标识符，多字符运算符 > 单字符

实现关键

本讲小结：词法分析器的实现要点

状态与转移：程序逻辑

- 状态表示：START, IDENT, NUMBER, OPERATOR 等
- 状态转移：根据当前字符决定下一个状态
- 核心原则：
 - 最长匹配：识别最长可能单词
 - 优先级：关键字 > 标识符，多字符运算符 > 单字符
 - 状态重置：识别完成或遇到分割点

实现关键

本讲小结：词法分析器的实现要点

状态与转移：程序逻辑

- 状态表示：START, IDENT, NUMBER, OPERATOR 等
- 状态转移：根据当前字符决定下一个状态
- 核心原则：
 - 最长匹配：识别最长可能单词
 - 优先级：关键字 > 标识符，多字符运算符 > 单字符
 - 状态重置：识别完成或遇到分割点

实现关键

- 优先处理关键字：先检查关键字，再检查标识符

本讲小结：词法分析器的实现要点

状态与转移：程序逻辑

- 状态表示：START, IDENT, NUMBER, OPERATOR 等
- 状态转移：根据当前字符决定下一个状态
- 核心原则：
 - 最长匹配：识别最长可能单词
 - 优先级：关键字 > 标识符，多字符运算符 > 单字符
 - 状态重置：识别完成或遇到分割点

实现关键

- 优先处理关键字：先检查关键字，再检查标识符
- 多字符运算符：如==, !=, <=, >=

本讲小结：词法分析器的实现要点

状态与转移：程序逻辑

- 状态表示：START, IDENT, NUMBER, OPERATOR 等
- 状态转移：根据当前字符决定下一个状态
- 核心原则：
 - 最长匹配：识别最长可能单词
 - 优先级：关键字 > 标识符，多字符运算符 > 单字符
 - 状态重置：识别完成或遇到分割点

实现关键

- 优先处理关键字：先检查关键字，再检查标识符
- 多字符运算符：如==, !=, <=, >=
- **状态重置**：识别完成后重置状态

本讲小结：词法分析器的实现要点

状态与转移：程序逻辑

- 状态表示：START, IDENT, NUMBER, OPERATOR 等
- 状态转移：根据当前字符决定下一个状态
- 核心原则：
 - 最长匹配：识别最长可能单词
 - 优先级：关键字 > 标识符，多字符运算符 > 单字符
 - 状态重置：识别完成或遇到分割点

实现关键

- 优先处理关键字：先检查关键字，再检查标识符
- 多字符运算符：如==, !=, <=, >=
- 状态重置：识别完成后重置状态
- 错误处理：非法字符、不完整字符串等

本讲小结：词法分析器的实现要点

状态与转移：程序逻辑

- 状态表示：START, IDENT, NUMBER, OPERATOR 等
- 状态转移：根据当前字符决定下一个状态
- 核心原则：
 - 最长匹配：识别最长可能单词
 - 优先级：关键字 > 标识符，多字符运算符 > 单字符
 - 状态重置：识别完成或遇到分割点

实现关键

- 优先处理关键字：先检查关键字，再检查标识符
- 多字符运算符：如==, !=, <=, >=
- 状态重置：识别完成后重置状态
- 错误处理：非法字符、不完整字符串等
- 编码处理：文件编码与程序内码的转换

作业题 2：字符串前缀对词法分析器的影响

- C++11 引入了多种字符串字面值前缀，如 `u8`、`u`、`U`、`R` 以及它们的组合 `u8R`、`uR`、`UR` 等。
- 请完成以下任务：
 - 简述这些前缀如何影响词法分析器的工作（对标识符识别的影响）
 - 描述如何正确处理带前缀的字符串常量
 - 绘制相关状态转移图，对比无此前缀时和增加前缀后的情况